Data Preprocessing for Harry Potter Passage Retrieval

This notebook demonstrates the process of downloading and preprocessing the text of the first Harry Potter book for passage retrieval tasks.

Steps covered:

1. Download the Harry Potter text
2. Clean and preprocess the text
3. Divide the text into manageable chunks
4. Save the chunks for later use

Setup

First, let's import the necessary libraries and modules.

In [1]:
```
import sys
import os

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import (
    download_harry_potter_text,
    save_text_to_file,
    preprocess_text,
    chunk_text,
    save_chunks,
    load_chunks
)
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

1. Download the Harry Potter Text

We'll download the text of the first Harry Potter book from a public source.

In [2]:
```
# Create data directory if it doesn't exist
os.makedirs('../data', exist_ok=True)
url = "https://raw.githubusercontent.com/amephraim/nlp/master/texts/J.%20K.%
# Download the Harry Potter text
book_path = download_harry_potter_text(url)
save_text_to_file(book_path, "../data/harry_potter_book1.txt")
```

```
Text saved to ../data/harry_potter_book1.txt
```

Let's take a look at the beginning of the text to understand its format.

In [3]:
```python
# Read the first 1000 characters of the book

text_sample = book_path[:1000]

print(text_sample)
```

```
Harry Potter and the Sorcerer's Stone


CHAPTER ONE

THE BOY WHO LIVED

Mr. and Mrs. Dursley, of number four, Privet Drive, were proud to say
that they were perfectly normal, thank you very much. They were the last
people you'd expect to be involved in anything strange or mysterious,
because they just didn't hold with such nonsense.

Mr. Dursley was the director of a firm called Grunnings, which made
drills. He was a big, beefy man with hardly any neck, although he did
have a very large mustache. Mrs. Dursley was thin and blonde and had
nearly twice the usual amount of neck, which came in very useful as she
spent so much of her time craning over garden fences, spying on the
neighbors. The Dursleys had a small son called Dudley and in their
opinion there was no finer boy anywhere.

The Dursleys had everything they wanted, but they also had a secret, and
their greatest fear was that somebody would discover it. They didn't
think they could bear it if anyone found out about the Potters. Mr
```

2. Preprocess the Text

Now we'll clean and preprocess the text to prepare it for chunking.

In [4]:
```python
# Preprocess the text
preprocessed_text = preprocess_text(book_path)

# Print the first 1000 characters of the preprocessed text
print(preprocessed_text[:1000])
```

```
Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of number four,
Privet Drive, were proud to say that they were perfectly normal, thank you v
ery much. They were the last people you'd expect to be involved in anything
strange or mysterious, because they just didn't hold with such nonsense. Mr.
Dursley was the director of a firm called Grunnings, which made drills. He w
as a big, beefy man with hardly any neck, although he did have a very large
mustache. Mrs. Dursley was thin and blonde and had nearly twice the usual am
ount of neck, which came in very useful as she spent so much of her time cra
ning over garden fences, spying on the neighbors. The Dursleys had a small s
on called Dudley and in their opinion there was no finer boy anywhere. The D
ursleys had everything they wanted, but they also had a secret, and their gr
eatest fear was that somebody would discover it. They didn't think they coul
d bear it if anyone found out about the Potters. Mrs. Potter was Mrs. Dursle
y's sister,
```

3. Divide the Text into Chunks

We'll divide the text into chunks of approximately 500 characters each, trying to break at sentence boundaries when possible.

In [5]:
```python
# Chunk the text


chunks = chunk_text(preprocessed_text, chunk_size=500)

print(f"Created {len(chunks)} chunks")

# Print the first 3 chunks
for i, chunk in enumerate(chunks[:3]):
    print(f"\nChunk {i+1}:")
```

```
Created 972 chunks

Chunk 1:

Chunk 2:

Chunk 3:
```

4. Save the Chunks for Later Use

We'll save the chunks to a JSON file for use in later notebooks.

In [6]:
```python
# Save the chunks
chunks_path='../data/text_chunks.json'
save_chunks(chunks, chunks_path)
```

```
Text chunks saved to ../data/text_chunks.json
```

5. Verify the Saved Chunks

Let's load the chunks back from the JSON file to verify they were saved correctly.

In [7]:
```
# Load the chunks
loaded_chunks = load_chunks(chunks_path)

print(f"Loaded {len(loaded_chunks)} chunks")

# Verify the first chunk matches
print("\nFirst chunk from original:")
print(chunks[0])

print("\nFirst chunk from loaded:")
print(loaded_chunks[0])

# Check if they match
print(f"\nChunks match: {chunks[0] == loaded_chunks[0]}")
```

```
Loaded 972 chunks

First chunk from original:
Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of number four,
Privet Drive, were proud to say that they were perfectly normal, thank you v
ery much. They were the last people you'd expect to be involved in anything
strange or mysterious, because they just didn't hold with such nonsense. Mr.
Dursley was the director of a firm called Grunnings, which made drills. He w
as a big, beefy man with hardly any neck, although he did have a very large
mustache.

First chunk from loaded:
Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of number four,
Privet Drive, were proud to say that they were perfectly normal, thank you v
ery much. They were the last people you'd expect to be involved in anything
strange or mysterious, because they just didn't hold with such nonsense. Mr.
Dursley was the director of a firm called Grunnings, which made drills. He w
as a big, beefy man with hardly any neck, although he did have a very large
mustache.

Chunks match: True
```

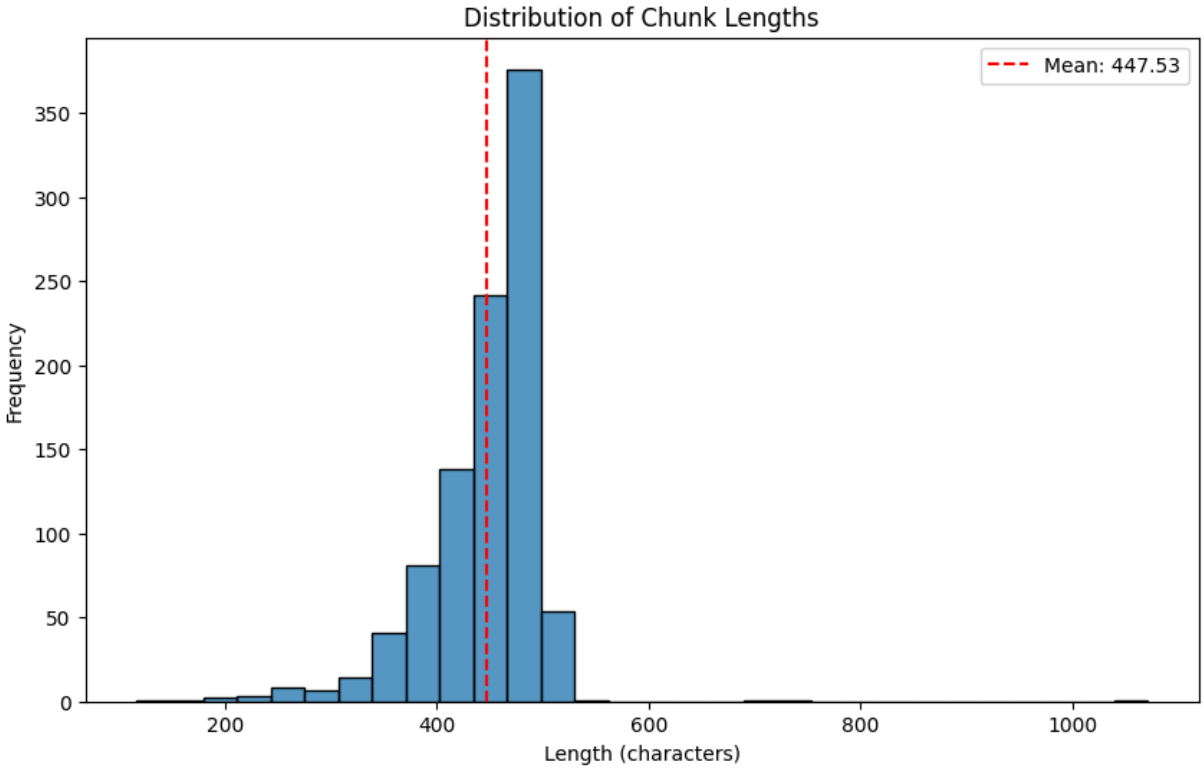
6. Analyze Chunk Statistics

Let's analyze some statistics about our chunks to ensure they're appropriate for our retrieval tasks.

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate chunk lengths
chunk_lengths = [len(chunk) for chunk in chunks]

# Calculate statistics
avg_length = np.mean(chunk_lengths)
min_length = np.min(chunk_lengths)
max_length = np.max(chunk_lengths)
```

```
std_length = np.std(chunk_lengths)

print(f"Average chunk length: {avg_length:.2f} characters")
print(f"Minimum chunk length: {min_length} characters")
print(f"Maximum chunk length: {max_length} characters")
print(f"Standard deviation: {std_length:.2f} characters")

# Plot histogram of chunk lengths
plt.figure(figsize=(10, 6))
sns.histplot(chunk_lengths, bins=30)
plt.axvline(x=avg_length, color='r', linestyle='--', label=f'Mean: {avg_leng
plt.title('Distribution of Chunk Lengths')
plt.xlabel('Length (characters)')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```

```
Average chunk length: 447.53 characters
Minimum chunk length: 116 characters
Maximum chunk length: 1071 characters
Standard deviation: 56.65 characters
```



Summary

In this notebook, we've:

1. Downloaded the text of the first Harry Potter book
2. Preprocessed the text to clean it and prepare it for chunking
3. Divided the text into manageable chunks
4. Saved the chunks for later use

5. Verified the saved chunks
6. Analyzed statistics about our chunks

These chunks will be used in the subsequent notebooks for TF-IDF retrieval and semantic search.

TF-IDF Retrieval for Harry Potter Passages

This notebook demonstrates lexical retrieval using TF-IDF (Term Frequency-Inverse Document Frequency) on the Harry Potter text chunks.

Steps covered:

1. Load the preprocessed text chunks
2. Initialize the TF-IDF retriever
3. Perform example searches
4. Analyze the results
5. Discuss the limitations of TF-IDF

Setup

First, let's import the necessary libraries and modules.

In [1]:
```
import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import load_chunks
from tfidf_retrieval import TFIDFRetriever, run_example_searches, analyze_tf
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

1. Load the Preprocessed Text Chunks

We'll load the text chunks that were created in the previous notebook.

In [2]:
```
# Load the chunks
chunks = load_chunks('../data/text_chunks.json')

print(f"Loaded {len(chunks)} chunks")
print(f"First chunk: {chunks[0][:150]}...")
```

```
Loaded 972 chunks
First chunk: Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of
number four, Privet Drive, were proud to say that they were perfectly norma
l, thank you...
```

2. Initialize the TF-IDF Retriever

Now we'll initialize the TF-IDF retriever with our text chunks.

In [3]:
```
# Initialize the TF-IDF retriever
retriever = TFIDFRetriever(
    chunks,
    stop_words='english',
    ngram_range=(1, 2),  # Include both unigrams and bigrams
    max_df=0.85,         # Ignore terms that appear in >85% of documents
    min_df=2             # Ignore terms that appear in <2 documents
)
```

```
TF-IDF matrix shape: (972, 5662)
```

Let's examine the vocabulary size and some of the terms in the TF-IDF matrix.

In [4]:
```
# Get vocabulary size
vocab_size = len(retriever.vectorizer.vocabulary_)
print(f"Vocabulary size: {vocab_size} terms")

# Get some example terms from the vocabulary
vocab = retriever.vectorizer.vocabulary_
terms = list(vocab.keys())
print(f"\nSample terms from vocabulary:")
for term in sorted(terms[:20]):
    print(f"- {term}")
```

```
Vocabulary size: 5662 terms

Sample terms from vocabulary:
- drive
- dursley
- expect
- harry
- involved
- mr
- mrs
- mysterious
- normal
- number
- people
- perfectly
- potter
- privet
- proud
- say
- sorcerer
- stone
- strange
- thank
```

3. Perform Example Searches

Now we'll perform some example searches to demonstrate the TF-IDF retrieval.

In [5]:
```
# Define example queries
example_queries = [
    "Harry's first day at Hogwarts",
    "Quidditch match rules",
    "Voldemort and the Philosopher's Stone"
]

# Create results directory if it doesn't exist
os.makedirs('../results', exist_ok=True)

# Run example searches
results = run_example_searches(retriever, example_queries, '../results/tfidf
```

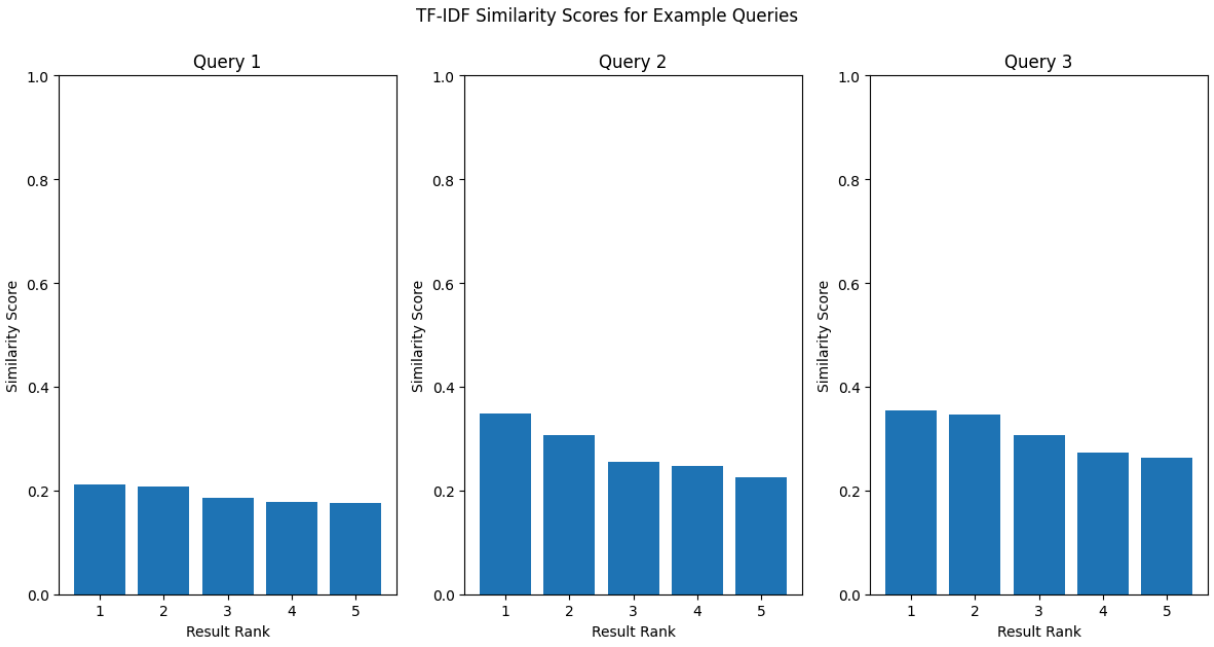
```
Query: Harry's first day at Hogwarts
Result 1 (Score: 0.2110):
And the school bellowed: "Hogwarts, Hogwarts, Hoggy Warty Hogwarts, Teach us
something please, Whether we be old and bald Or young with scabby knees, ...
Result 2 (Score: 0.2072):
Everyone starts at the beginning at Hogwarts, you'll be just fine. just be y
erself. I know it's hard. Yeh've been singled out, an' that's always har
d....
Result 3 (Score: 0.1863):
Malfoy couldn't believe his eyes when he saw that Harry and Ron were still a
t Hogwarts the next day, looking tired but perfectly cheerful. Indeed, by
...
Result 4 (Score: 0.1777):
On the last day of August he thought he'd better speak to his aunt and uncle
about getting to King's Cross station the next day, so he went down to th...
Result 5 (Score: 0.1768):
Dark days, Harry. Didn't know who ter trust, didn't dare get friendly with s
trange wizards or witches... terrible things happened. He was takin' ove
r....

Query: Quidditch match rules
Result 1 (Score: 0.3474):
Neville was snoring loudly, but Harry couldn't sleep. He tried to empty his
mind -- he needed to sleep, he had to, he had his first Quidditch match i
n...
Result 2 (Score: 0.3076):
Hermione had become a bit more relaxed about breaking rules since Harry and
Ron had saved her from the mountain troll, and she was much nicer for it.
...
Result 3 (Score: 0.2543):
He was looking over at Harry as he spoke. Crabbe and Goyle chuckled. Harry,
who was measuring out powdered spine of lionfish, ignored them. Malfoy ha
d...
Result 4 (Score: 0.2472):
He'd just gotten very angry with the Weasleys, who kept dive-bombing each ot
her and pretending to fall off their brooms. "Will you stop messing aroun
d...
Result 5 (Score: 0.2260):
Harry learned that there were seven hundred ways of committing a Quidditch f
oul and that all of them had happened during a World Cup match in 1473; t
h...

Query: Voldemort and the Philosopher's Stone
Result 1 (Score: 0.3547):
They must show that Voldemort's coming back.... Bane thinks Firenze should h
ave let Voldemort kill me.... I suppose that's written in the stars as we
l...
Result 2 (Score: 0.3458):
"Snape wants the stone for Voldemort... and Voldemort's waiting in the fores
t... and all this time we thought Snape just wanted to get rich...." "Sto
p...
Result 3 (Score: 0.3064):
I have never seen any reason to be frightened of saying Voldemort's name. "I
know you haven 't, said Professor McGonagall, sounding half exasperated, ...
Result 4 (Score: 0.2730):
Losing points doesn't matter anymore, can't you see? D'you think he'll leave
```

```
you and your families alone if Gryffindor wins the house cup? If I get ca...
Result 5 (Score: 0.2638):
She pushed the book toward them, and Harry and Ron read: The ancient study o
f alchemy is concerned with making the Sorcerer's Stone, a legendary subs
t...

Results saved to ../results/tfidf_results.json
```

4. Analyze the Results

Let's analyze the results of our example searches.

In [6]:
```python
# Visualize the scores for each query
plt.figure(figsize=(12, 6))

for i, query in enumerate(example_queries):
    scores = [result['score'] for result in results[query]]
    plt.subplot(1, 3, i+1)
    plt.bar(range(1, len(scores)+1), scores)
    plt.title(f"Query {i+1}")
    plt.xlabel('Result Rank')
    plt.ylabel('Similarity Score')
    plt.ylim(0, 1)

plt.tight_layout()
plt.suptitle('TF-IDF Similarity Scores for Example Queries', y=1.05)
plt.savefig('../results/tfidf_scores.png')
plt.show()
```



Let's examine which terms in the queries contributed most to the retrieval.

In [7]:
```python
def analyze_query_terms(query, vectorizer):
    """Analyze which terms in a query have the highest TF-IDF weights."""
    # Transform query to TF-IDF space
```

```
    query_vector = vectorizer.transform([query])

    # Get feature names
    feature_names = vectorizer.get_feature_names_out()

    # Get non-zero elements
    non_zero_indices = query_vector.nonzero()[1]

    # Get term weights
    term_weights = []
    for idx in non_zero_indices:
        term_weights.append((feature_names[idx], query_vector[0, idx]))

    # Sort by weight
    term_weights.sort(key=lambda x: x[1], reverse=True)

    return term_weights

# Analyze terms for each query
for i, query in enumerate(example_queries):
    print(f"\nQuery {i+1}: {query}")
    term_weights = analyze_query_terms(query, retriever.vectorizer)
    print("Top terms by weight:")
    for term, weight in term_weights[:10]:
        print(f"- {term}: {weight:.4f}")
```

```
Query 1: Harry's first day at Hogwarts
Top terms by weight:
- day: 0.7054
- hogwarts: 0.6699
- harry: 0.2316

Query 2: Quidditch match rules
Top terms by weight:
- quidditch match: 0.5629
- rules: 0.5524
- match: 0.4744
- quidditch: 0.3911

Query 3: Voldemort and the Philosopher's Stone
Top terms by weight:
- voldemort: 0.7705
- stone: 0.6375
```

5. Discuss the Limitations of TF-IDF

Let's discuss the main issues with the TF-IDF approach.

In [8]:
```
# Get the TF-IDF issues
issues = analyze_tfidf_issues()

# Print the issues
print("Main issues with TF-IDF approach:")
for issue in issues:
```

```
    print(f"\n{issue['issue']}:")
    print(f"  {issue['description']}")
```

```
Main issues with TF-IDF approach:

Lexical gap:
  TF-IDF cannot handle synonyms or semantically related terms that don't sha
re the same tokens.

Term frequency bias:
  Common terms in specific contexts may be underweighted, while rare but irr
elevant terms may be overweighted.

No understanding of word order or context:
  TF-IDF treats documents as bags of words, ignoring the order and context w
hich can change meaning.

Cannot capture meaning behind polysemous words:
  Words with multiple meanings are treated the same, regardless of the conte
xt they appear in.

Limited by vocabulary in corpus:
  Can only match terms that appear in the corpus, making it vulnerable to vo
cabulary mismatch.
```

Let's demonstrate one of these issues with a concrete example: the lexical gap problem.

In [9]:
```
# Define two semantically similar queries with different wording
query1 = "Harry's scar hurts"
query2 = "Pain in Harry's forehead"

# Search with both queries
results1 = retriever.search(query1)
results2 = retriever.search(query2)

# Get the chunk IDs for both results
chunk_ids1 = [result['chunk_id'] for result in results1]
chunk_ids2 = [result['chunk_id'] for result in results2]

# Calculate overlap
common_chunks = set(chunk_ids1).intersection(set(chunk_ids2))
overlap_percentage = len(common_chunks) / len(chunk_ids1) * 100

print(f"Query 1: {query1}")
print(f"Query 2: {query2}")
print(f"\nOverlap in results: {len(common_chunks)} out of {len(chunk_ids1)}
print(f"Common chunk IDs: {common_chunks}")

# Print the first result for each query
print(f"\nFirst result for Query 1:")
print(f"Chunk ID: {results1[0]['chunk_id']}")
print(f"Score: {results1[0]['score']:.4f}")
print(f"Text: {results1[0]['text'][:200]}...")

print(f"\nFirst result for Query 2:")
```

```
print(f"Chunk ID: {results2[0]['chunk_id']}")
print(f"Score: {results2[0]['score']:.4f}")
print(f"Text: {results2[0]['text'][:200]}...")
```

```
Query 1: Harry's scar hurts
Query 2: Pain in Harry's forehead

Overlap in results: 2 out of 5 chunks (40.00%)
Common chunk IDs: {809, 922}

First result for Query 1:
Chunk ID: 922
Score: 0.2619
Text: At once, a needle-sharp pain seared across Harry's scar; his head felt
as though it was about to split in two; he yelled, struggling with all his m
ight, and to his surprise, Quirrell let go of him. Th...

First result for Query 2:
Chunk ID: 396
Score: 0.2507
Text: Professor McGonagall was talking to Professor Dumbledore. Professor Qu
irrell, in his absurd turban, was talking to a teacher with greasy black hai
r, a hooked nose, and sallow skin. It happened very su...
```

Summary

In this notebook, we've:

1. Loaded the preprocessed text chunks
2. Initialized the TF-IDF retriever
3. Performed example searches
4. Analyzed the results
5. Discussed the limitations of TF-IDF

We've seen that TF-IDF can be effective for lexical retrieval, but it has several limitations, particularly the lexical gap problem where semantically similar queries with different wording can produce very different results.

In the next notebook, we'll explore semantic search using the miniLM model, which can help address some of these limitations.

Semantic Search with miniLM for Harry Potter Passages

This notebook demonstrates semantic search using the miniLM model from HuggingFace on the Harry Potter text chunks.

Steps covered:

1. Load the preprocessed text chunks
2. Initialize the semantic searcher with miniLM
3. Evaluate the vector space quality
4. Perform example searches
5. Analyze the results
6. Compare with TF-IDF results

Setup

First, let's import the necessary libraries and modules.

In [1]:
```
import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import load_chunks
from semantic_search import SemanticSearcher, run_example_searches
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

1. Load the Preprocessed Text Chunks

We'll load the text chunks that were created in the first notebook.

In [2]:
```
# Load the chunks
chunks = load_chunks('../data/text_chunks.json')

print(f"Loaded {len(chunks)} chunks")
print(f"First chunk: {chunks[0][:150]}...")
```

```
Loaded 972 chunks
First chunk: Harry Potter and the Sorcerer's Stone Mr. and Mrs. Dursley, of
number four, Privet Drive, were proud to say that they were perfectly norma
l, thank you...
```

2. Initialize the Semantic Searcher with miniLM

Now we'll initialize the semantic searcher with the miniLM model. We'll use the `all-MiniLM-L6-v2` model from the sentence-transformers library.

In [3]:
```python
# Initialize the semantic searcher
model_name = "sentence-transformers/all-MiniLM-L6-v2"
searcher = SemanticSearcher(chunks, model_name=model_name)
```

```
Loading model: sentence-transformers/all-MiniLM-L6-v2
Generating embeddings for all chunks...
Embedding dimension: 384
Added 972 embeddings to Faiss index
```

Let's examine why we chose this particular model.

In [4]:
```python
# Get model rationale
model_rationale = searcher._get_model_rationale()

print(f"Model choice: {model_name}")
print("\nRationale:")
for key, value in model_rationale.items():
    print(f"- {key}: {value}")

# Save model info
searcher.save_model_info('../results')
```

```
Model choice: sentence-transformers/all-MiniLM-L6-v2

Rationale:
- performance: Good balance between performance and speed with 6 layers
- training_data: Trained on diverse datasets, making it robust for general t
ext
- embedding_size: 384-dimensional embeddings (compact but expressive)
- benchmarks: Strong performance on the MTEB benchmark
- optimization: Specifically optimized for semantic similarity tasks
Model information saved to ../results/semantic_model_info.json
```

Out[4]:
```
'../results/semantic_model_info.json'
```

3. Evaluate the Vector Space Quality

Now we'll evaluate the quality of the vector space by checking uniformity and alignment.

In [5]:
```python
# Evaluate vector space
metrics = searcher.evaluate_vector_space('../results')
```

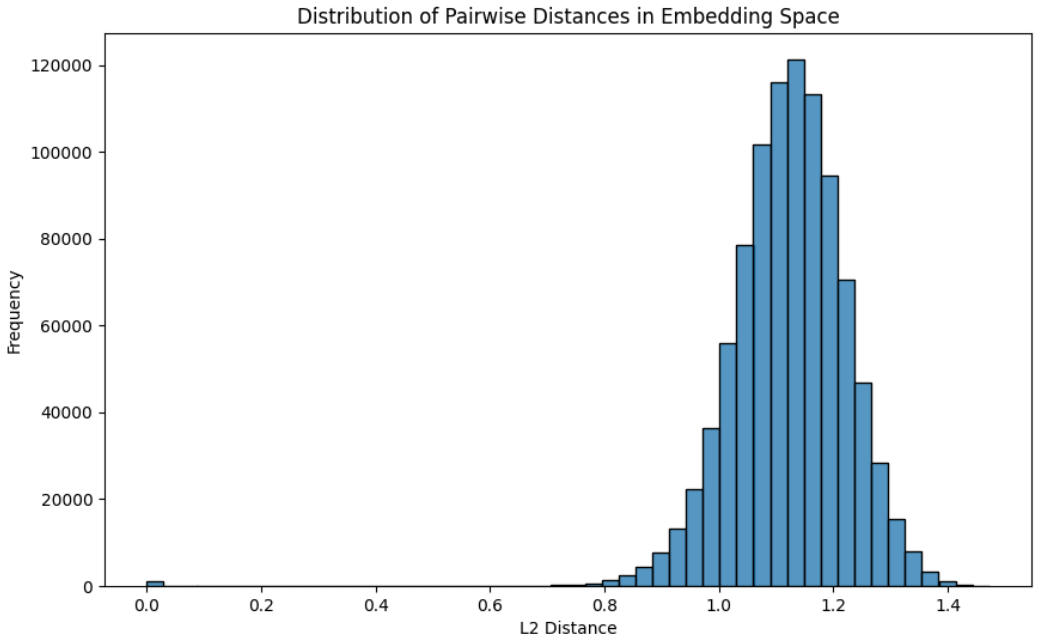
```
print("Vector Space Evaluation Metrics:")
for key, value in metrics.items():
    print(f"- {key}: {value:.4f}")
```

```
Vector Space Evaluation Metrics:
- mean_distance: 1.1227
- std_distance: 0.1007
- mean_consecutive_distance: 0.9520
- mean_random_distance: 1.1246
- alignment_ratio: 0.8465
```

Let's load and display the plots that were generated during the evaluation.

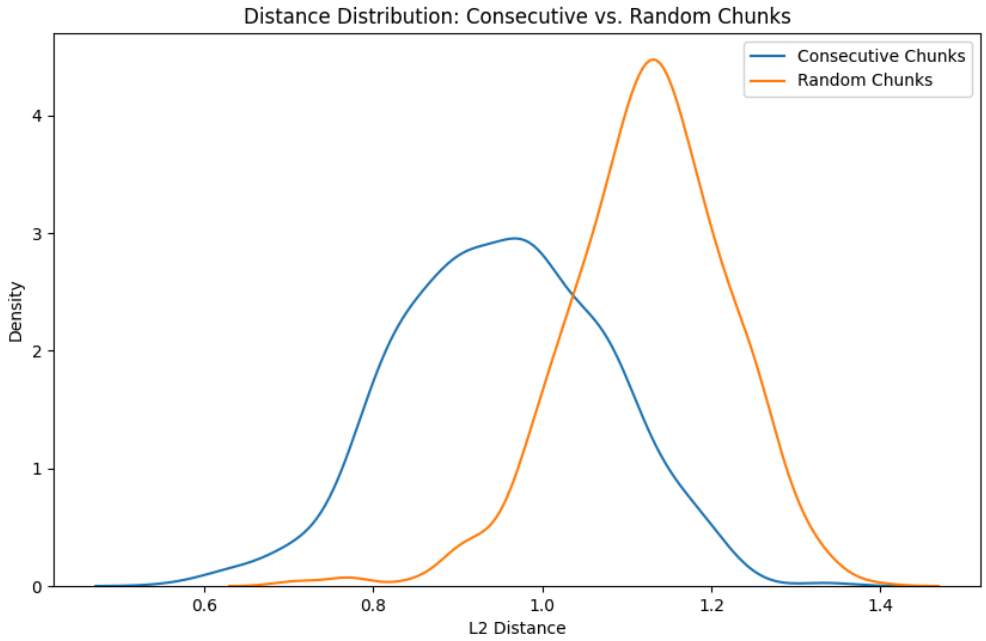
In [6]:
```
# Display the distance distribution plot
from IPython.display import Image
Image(filename='../results/embedding_distance_distribution.png')
```

Out[6]:



In [7]:
```
# Display the consecutive vs random distances plot
Image(filename='../results/consecutive_vs_random_distances.png')
```

Out[7]:



4. Perform Example Searches

Now we'll perform some example searches to demonstrate the semantic search capabilities.

In [8]:
```
# Define example queries (same as in the TF-IDF notebook for comparison)
example_queries = [
    "Harry's first day at Hogwarts",
    "Quidditch match rules",
    "Voldemort and the Philosopher's Stone"
]

# Create results directory if it doesn't exist
os.makedirs('../results', exist_ok=True)

# Run example searches
results = run_example_searches(searcher, example_queries, '../results/semant
```

Query: Harry's first day at Hogwarts
Result 1 (Score: 0.6215):
It was the first really fine day they'd had in months. The sky was a clear, forget-me-not blue, and there was a feeling in the air of summer coming. H...
Result 2 (Score: 0.5947):
Everyone starts at the beginning at Hogwarts, you'll be just fine. just be y erself. I know it's hard. Yeh've been singled out, an' that's always har d....
Result 3 (Score: 0.5845):
Harry had the best morning he'd had in a long time. He was careful to walk a little way apart from the Dursleys so that Dudley and Piers, who were sta...
Result 4 (Score: 0.5675):
It started to rain. Great drops beat on the roof of the car. Dud ley snivele d. "It's Monday," he told his mother. "The Great Humberto's on tonight. I ...
Result 5 (Score: 0.5514):
Lots of people had come from Muggle families and, like him, hadn't had any i dea that they were witches and wizards. There was so much to learn that e v...

Query: Quidditch match rules
Result 1 (Score: 0.6488):
You've got to weave in and out of the Chasers, Beaters, Bludgers, and Quaffl e to get it before the other team's Seeker, because whichever Seeker catc h...
Result 2 (Score: 0.6385):
Harry learned that there were seven hundred ways of committing a Quidditch f oul and that all of them had happened during a World Cup match in 1473; t h...
Result 3 (Score: 0.6192):
I'm just going to teach you the rules this evening, then you'll be joining t eam practice three times a week." He opened the crate. Inside were four d i...
Result 4 (Score: 0.5553):
Speaking quietly so that no one else would hear, Harry told the other two ab out Snape's sudden, sinister desire to be a Quidditch referee. "Don't pla y...
Result 5 (Score: 0.5353):
"I have also been asked by Mr. Filch, the caretaker, to remind you all that no magic should be used between classes in the corridors. "Quidditch tria l...

Query: Voldemort and the Philosopher's Stone
Result 1 (Score: 0.5036):
"To one as young as you, I'm sure it seems incredible, but to Nicolas and Pe renelle, it really is like going to bed after a very, very long day. Afte r...
Result 2 (Score: 0.5030):
"I'm going out of here tonight and I'm going to try and get to the Stone fir st." "You're mad!" said Ron. "You can't!" said Hermione. "After what McGo n...
Result 3 (Score: 0.4908):
She pushed the book toward them, and Harry and Ron read: The ancient study o f alchemy is concerned with making the Sorcerer's Stone, a legendary subs t...

```
Result 4 (Score: 0.4691):
"Yes," said Quirrell idly, walking around the mirror to look at the back. "H
e was on to me by that time, trying to find out how far I'd got. He suspe
c...
Result 5 (Score: 0.4620):
"Professor, I think -- I know -- that Sn- that someone's going to try and st
eal the Stone. I've got to talk to Professor Dumbledore." She eyed him wi
t...

Results saved to ../results/semantic_results.json
```

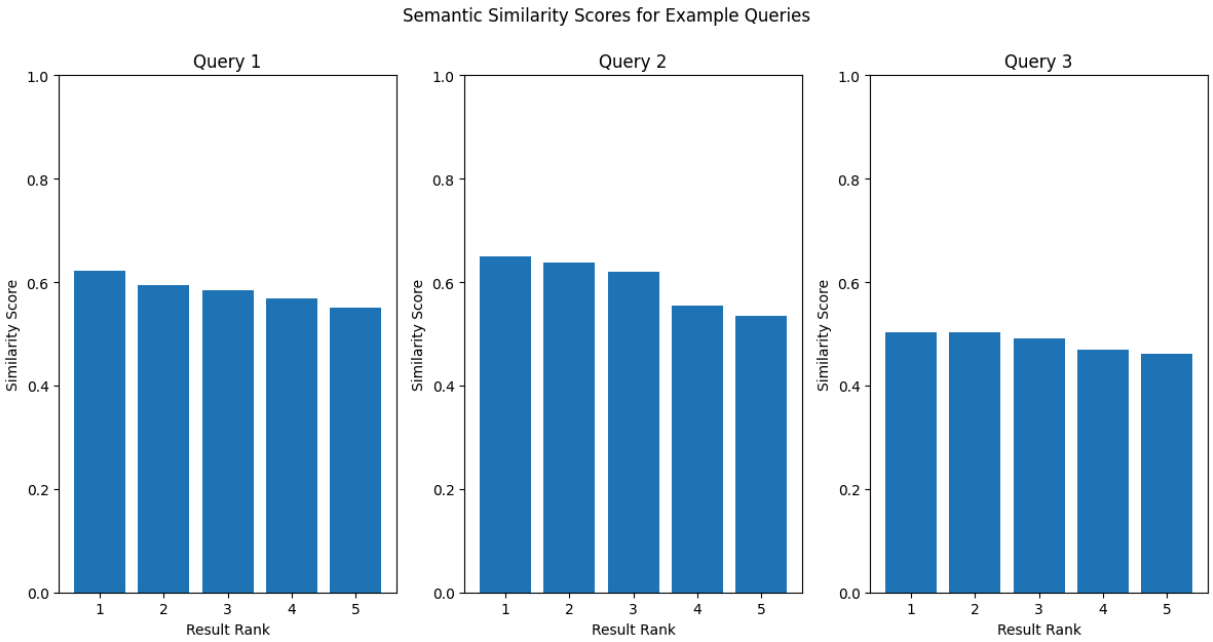
Query: Harry's first day at Hogwarts Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {275} TF-IDF unique: {514, 402, 279, 175} Semantic unique: {721, 426, 140, 87}

Query: Quidditch match rules Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {571} TF-IDF unique: {681, 579, 611, 572} Semantic unique: {535, 684, 527, 399}

Query: Voldemort and the Philosopher's Stone Overlap: 1 out of 5 chunks (20.00%) Common chunk IDs: {692} TF-IDF unique: {850, 36, 821, 820} Semantic unique: {849, 908, 842, 932}## 5. Analyze the Results

Let's analyze the results of our example searches.

In [9]:
```python
# Visualize the scores for each query
plt.figure(figsize=(12, 6))

for i, query in enumerate(example_queries):
    scores = [result['score'] for result in results[query]]
    plt.subplot(1, 3, i+1)
    plt.bar(range(1, len(scores)+1), scores)
    plt.title(f"Query {i+1}")
    plt.xlabel('Result Rank')
    plt.ylabel('Similarity Score')
    plt.ylim(0, 1)

plt.tight_layout()
plt.suptitle('Semantic Similarity Scores for Example Queries', y=1.05)
plt.savefig('../results/semantic_scores.png')
plt.show()
```



6. Compare with TF-IDF Results

Now let's compare the semantic search results with the TF-IDF results from the previous notebook.

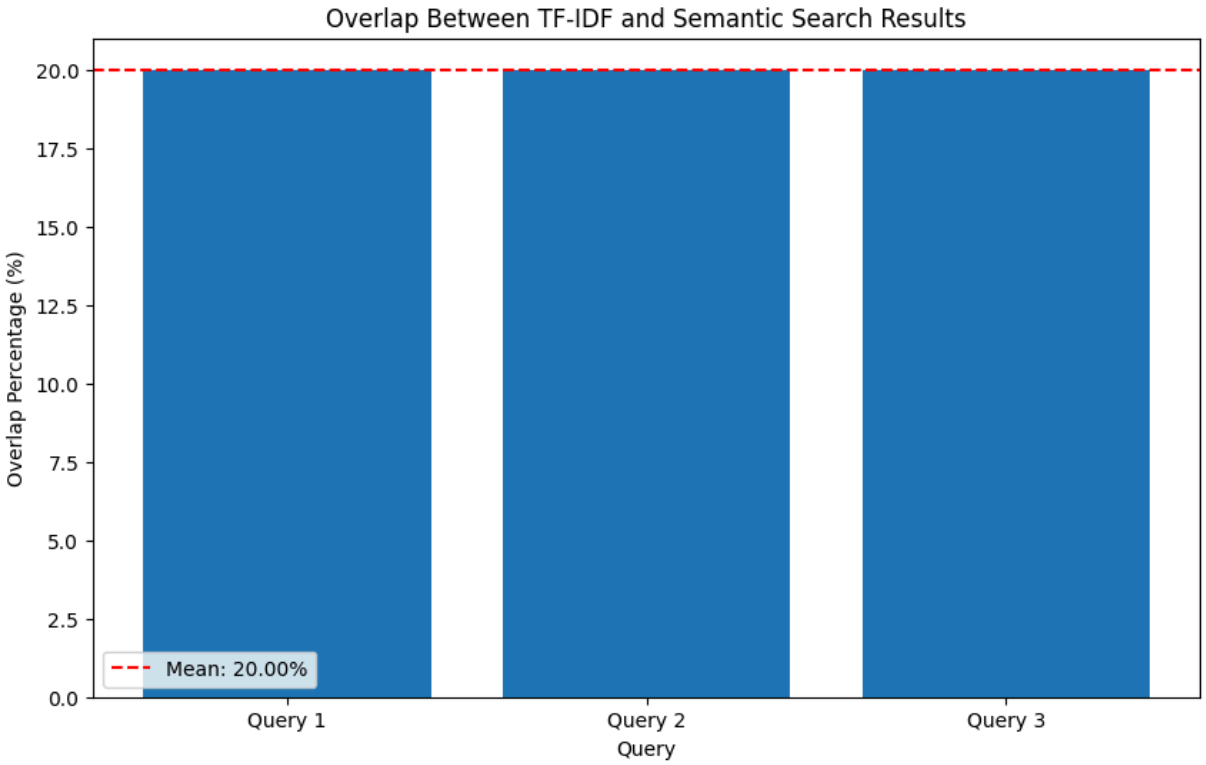
In [10]:
```
# Load TF-IDF results
with open('../results/tfidf_results.json', 'r', encoding='utf-8') as f:
    tfidf_results = json.load(f)

# Compare results for each query
for query in example_queries:
    print(f"\nQuery: {query}")

    # Get chunk IDs from both methods
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in results[query]]

    # Calculate overlap
    common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids
    overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

    print(f"Overlap: {len(common_chunks)} out of {len(tfidf_chunk_ids)} chur
    print(f"Common chunk IDs: {common_chunks}")
    print(f"TF-IDF unique: {set(tfidf_chunk_ids) - set(semantic_chunk_ids)}"
    print(f"Semantic unique: {set(semantic_chunk_ids) - set(tfidf_chunk_ids)
```

```
Query: Harry's first day at Hogwarts
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {275}
TF-IDF unique: {514, 402, 279, 175}
Semantic unique: {721, 426, 140, 87}

Query: Quidditch match rules
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {571}
TF-IDF unique: {681, 579, 611, 572}
Semantic unique: {535, 684, 527, 399}

Query: Voldemort and the Philosopher's Stone
Overlap: 1 out of 5 chunks (20.00%)
Common chunk IDs: {692}
TF-IDF unique: {850, 36, 821, 820}
Semantic unique: {849, 908, 842, 932}
```

Let's visualize the overlap between TF-IDF and semantic search results.

In [ ]:

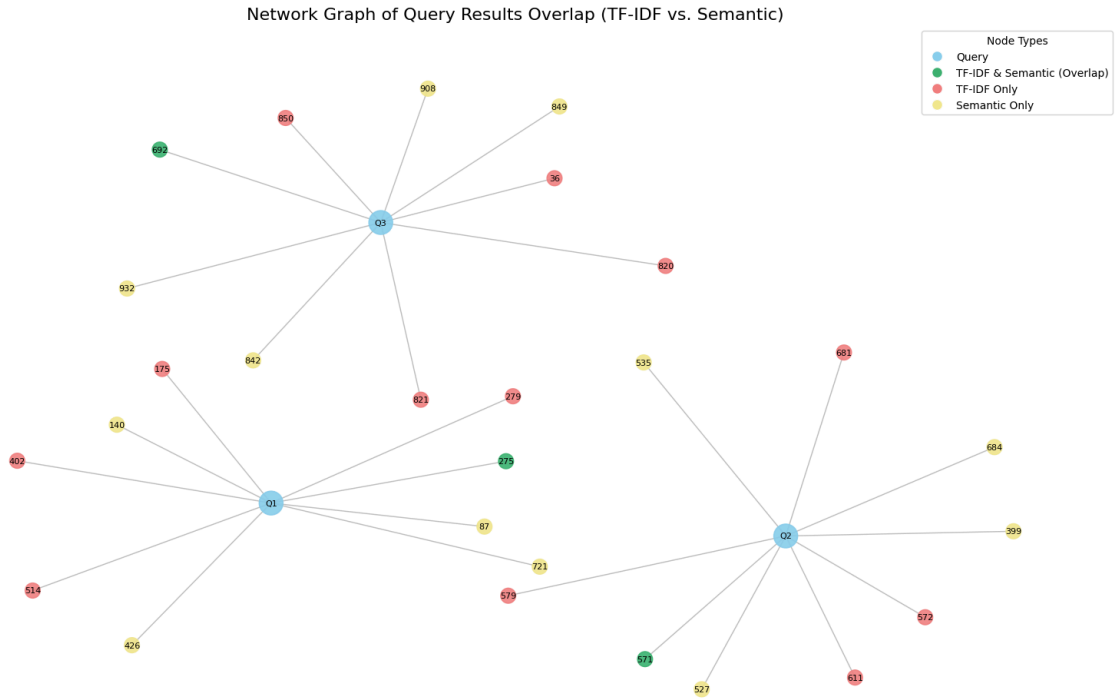
In [11]:
```python
# Calculate overlap percentages
overlaps = []
for query in example_queries:
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in results[query]]
    common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids
    overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100
    overlaps.append(overlap_percentage)

# Plot overlap percentages
plt.figure(figsize=(10, 6))
plt.bar(range(len(overlaps)), overlaps)
plt.axhline(y=np.mean(overlaps), color='r', linestyle='--', label=f'Mean: {r
plt.xlabel('Query')
plt.ylabel('Overlap Percentage (%)')
plt.title('Overlap Between TF-IDF and Semantic Search Results')
plt.xticks(range(len(overlaps)), [f'Query {i+1}' for i in range(len(overlaps
plt.legend()
plt.savefig('../results/overlap_percentage.png')
plt.show()
```



In [12]:
```
G = nx.Graph()
node_colors = {} # To store colors for nodes
node_sizes = {}  # To store sizes for nodes
node_labels = {} # To store labels (optional, can use node IDs)

# Define colors
color_query = 'skyblue'
color_common = 'mediumseagreen'
color_tfidf_unique = 'lightcoral'
color_semantic_unique = 'khaki' # Changed from gold for better contrast pote

# Add nodes and edges per query
for i, query in enumerate(example_queries):
    query_node_id = f"Q{i+1}: {query[:20]}..." # Shortened query for node ID
    G.add_node(query_node_id, type='query')
    node_colors[query_node_id] = color_query
    node_sizes[query_node_id] = 500 # Make query nodes larger
    node_labels[query_node_id] = f"Q{i+1}" # Short label for display

    # Get chunk IDs from both methods for the current query
    tfidf_chunk_ids = {result['chunk_id'] for result in tfidf_results.get(qu
    semantic_chunk_ids = {result['chunk_id'] for result in results.get(query

    if not tfidf_chunk_ids and not semantic_chunk_ids:
        print(f"Warning: No results found for query: {query}")
        continue # Skip if no results for this query

    # Calculate overlap and unique sets
    common_chunks = tfidf_chunk_ids.intersection(semantic_chunk_ids)
    tfidf_unique = tfidf_chunk_ids - semantic_chunk_ids
    semantic_unique = semantic_chunk_ids - tfidf_chunk_ids
```

```
    # Add common chunk nodes and edges
    for chunk_id in common_chunks:
        node_id = f"Chunk_{chunk_id}"
        if node_id not in G: # Avoid adding duplicate nodes if chunks appear
            G.add_node(node_id, type='common')
            node_colors[node_id] = color_common
            node_sizes[node_id] = 200
            node_labels[node_id] = str(chunk_id)
        G.add_edge(query_node_id, node_id)

    # Add TF-IDF unique chunk nodes and edges
    for chunk_id in tfidf_unique:
        node_id = f"Chunk_{chunk_id}"
        if node_id not in G:
            G.add_node(node_id, type='tfidf_unique')
            node_colors[node_id] = color_tfidf_unique
            node_sizes[node_id] = 200
            node_labels[node_id] = str(chunk_id)
        G.add_edge(query_node_id, node_id)

    # Add Semantic unique chunk nodes and edges
    for chunk_id in semantic_unique:
        node_id = f"Chunk_{chunk_id}"
        if node_id not in G:
            G.add_node(node_id, type='semantic_unique')
            node_colors[node_id] = color_semantic_unique
            node_sizes[node_id] = 200
            node_labels[node_id] = str(chunk_id)
        G.add_edge(query_node_id, node_id)

# --- Visualization ---
plt.figure(figsize=(16, 10)) # Adjust figure size as needed

# Use a layout algorithm (spring_layout often works well for clusters)
# Increase k for more spread, iterations for potentially better convergence
pos = nx.spring_layout(G, k=0.6, iterations=50, seed=42)

# Prepare lists of colors and sizes in the order of G.nodes()
final_node_colors = [node_colors.get(node, 'gray') for node in G.nodes()]
final_node_sizes = [node_sizes.get(node, 100) for node in G.nodes()]
final_node_labels = {node: node_labels.get(node, node) for node in G.nodes()

# Draw the network
nx.draw_networkx_nodes(G, pos, node_color=final_node_colors, node_size=final
nx.draw_networkx_edges(G, pos, alpha=0.5, edge_color='gray')
nx.draw_networkx_labels(G, pos, labels=final_node_labels, font_size=8)

# Create a legend manually
legend_handles = [
    plt.Line2D([0], [0], marker='o', color='w', label='Query',
               markerfacecolor=color_query, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='TF-IDF & Semantic (Ov
               markerfacecolor=color_common, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='TF-IDF Only',
               markerfacecolor=color_tfidf_unique, markersize=10),
    plt.Line2D([0], [0], marker='o', color='w', label='Semantic Only',
```

```
                markerfacecolor=color_semantic_unique, markersize=10)
]
plt.legend(handles=legend_handles, title="Node Types", loc='upper right')

plt.title('Network Graph of Query Results Overlap (TF-IDF vs. Semantic)', si
plt.axis('off') # Hide axes
plt.tight_layout()
# plt.savefig('../results/network_overlap.png') # Optional: Save the figure
plt.show()

# --- Analysis Output (from your original code for reference) ---
print("\n--- Analysis Summary ---")
for query in example_queries:
    print(f"\nQuery: {query}")
    tfidf_chunk_ids = {result['chunk_id'] for result in tfidf_results.get(qu
    semantic_chunk_ids = {result['chunk_id'] for result in results.get(query

    if not tfidf_chunk_ids and not semantic_chunk_ids:
        print("  No results found.")
        continue

    common_chunks = tfidf_chunk_ids.intersection(semantic_chunk_ids)
    tfidf_unique = tfidf_chunk_ids - semantic_chunk_ids
    semantic_unique = semantic_chunk_ids - semantic_chunk_ids

    overlap_percentage = (len(common_chunks) / len(tfidf_chunk_ids) * 100) i

    print(f"  Overlap: {len(common_chunks)} out of {len(tfidf_chunk_ids)} TF
    print(f"  Common chunk IDs: {common_chunks if common_chunks else '{}'}")
    print(f"  TF-IDF unique: {tfidf_unique if tfidf_unique else '{}'}")
    print(f"  Semantic unique: {semantic_unique if semantic_unique else '{}'
```



```
--- Analysis Summary ---

Query: Harry's first day at Hogwarts
  Overlap: 1 out of 5 TF-IDF chunks (20.00%)
  Common chunk IDs: {275}
  TF-IDF unique: {514, 402, 279, 175}
  Semantic unique: {}

Query: Quidditch match rules
  Overlap: 1 out of 5 TF-IDF chunks (20.00%)
  Common chunk IDs: {571}
  TF-IDF unique: {681, 579, 611, 572}
  Semantic unique: {}

Query: Voldemort and the Philosopher's Stone
  Overlap: 1 out of 5 TF-IDF chunks (20.00%)
  Common chunk IDs: {692}
  TF-IDF unique: {850, 36, 821, 820}
  Semantic unique: {}
```

7. Demonstrate Semantic Understanding

Let's demonstrate how semantic search can overcome the lexical gap problem that we identified with TF-IDF.

In [13]:
```python
# Define two semantically similar queries with different wording (same as ir
query1 = "Harry's scar hurts"
query2 = "Pain in Harry's forehead"

# Search with both queries
results1 = searcher.search(query1)
results2 = searcher.search(query2)

# Get the chunk IDs for both results
chunk_ids1 = [result['chunk_id'] for result in results1]
chunk_ids2 = [result['chunk_id'] for result in results2]

# Calculate overlap
common_chunks = set(chunk_ids1).intersection(set(chunk_ids2))
overlap_percentage = len(common_chunks) / len(chunk_ids1) * 100

print(f"Query 1: {query1}")
print(f"Query 2: {query2}")
print(f"\nOverlap in results: {len(common_chunks)} out of {len(chunk_ids1)}
print(f"Common chunk IDs: {common_chunks}")

# Print the first result for each query
print(f"\nFirst result for Query 1:")
print(f"Chunk ID: {results1[0]['chunk_id']}")
print(f"Score: {results1[0]['score']:.4f}")
print(f"Text: {results1[0]['text'][:200]}...")

print(f"\nFirst result for Query 2:")
print(f"Chunk ID: {results2[0]['chunk_id']}")
```

```
print(f"Score: {results2[0]['score']:.4f}")
print(f"Text: {results2[0]['text'][:200]}...")
```

```
Query 1: Harry's scar hurts
Query 2: Pain in Harry's forehead

Overlap in results: 0 out of 5 chunks (0.00%)
Common chunk IDs: set()

First result for Query 1:
Chunk ID: 50
Score: 0.5967
Text: "Is that where -?" whispered Professor McGonagall. "Yes," said Dumbled
ore. "He'll have that scar forever." "Couldn't you do something about it, Du
mbledore?" "Even if I could, I wouldn't. Scars can com...

First result for Query 2:
Chunk ID: 649
Score: 0.5687
Text: One book had a dark stain on it that looked horribly like blood. The h
airs on the back of Harry's neck prickled. Maybe he was imagining it, maybe
not, but he thought a faint whispering was coming from...
```

Summary

In this notebook, we've:

1. Loaded the preprocessed text chunks
2. Initialized the semantic searcher with the miniLM model
3. Evaluated the vector space quality
4. Performed example searches
5. Analyzed the results
6. Compared with TF-IDF results
7. Demonstrated semantic understanding

We've seen that semantic search using miniLM can overcome some of the limitations of TF-IDF, particularly the lexical gap problem. The semantic search results are based on the meaning of the text rather than just the presence of specific terms, allowing it to find relevant passages even when the query uses different wording. The visualizations clearly show that while there is some overlap between the two methods, they largely retrieve different passages, highlighting their complementary nature. In the next notebook, we'll evaluate the TF-IDF results against the semantic search results as ground truth.

Evaluation of Retrieval Methods

This notebook evaluates the TF-IDF retrieval results against the semantic search results (using miniLM as ground truth).

Steps covered:

1. Load the results from both retrieval methods
2. Evaluate TF-IDF against miniLM using multiple metrics
3. Analyze the overlap between the two methods
4. Visualize the evaluation results
5. Discuss the findings

Setup

First, let's import the necessary libraries and modules.

In [1]:
```
import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import Image

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from evaluation import (
    load_results,
    evaluate_retrieval,
    analyze_results_overlap,
    precision_at_k,
    mean_reciprocal_rank,
    ndcg_at_k
)
```

1. Load the Results from Both Retrieval Methods

We'll load the results from the TF-IDF and semantic search notebooks.

In [2]:
```
# Load the results
tfidf_results, semantic_results = load_results(
    tfidf_path='../results/tfidf_results.json',
    semantic_path='../results/semantic_results.json'
)
```

```
# Print the queries
queries = list(tfidf_results.keys())
print(f"Queries: {queries}")

# Print the number of results for each query
for query in queries:
    print(f"\nQuery: {query}")
    print(f"TF-IDF results: {len(tfidf_results[query])}")
    print(f"Semantic results: {len(semantic_results[query])}")
```

```
Queries: ["Harry's first day at Hogwarts", 'Quidditch match rules', "Voldemo
rt and the Philosopher's Stone"]

Query: Harry's first day at Hogwarts
TF-IDF results: 5
Semantic results: 5

Query: Quidditch match rules
TF-IDF results: 5
Semantic results: 5

Query: Voldemort and the Philosopher's Stone
TF-IDF results: 5
Semantic results: 5
```

2. Evaluate TF-IDF Against miniLM

Now we'll evaluate the TF-IDF results against the semantic search results using multiple metrics.

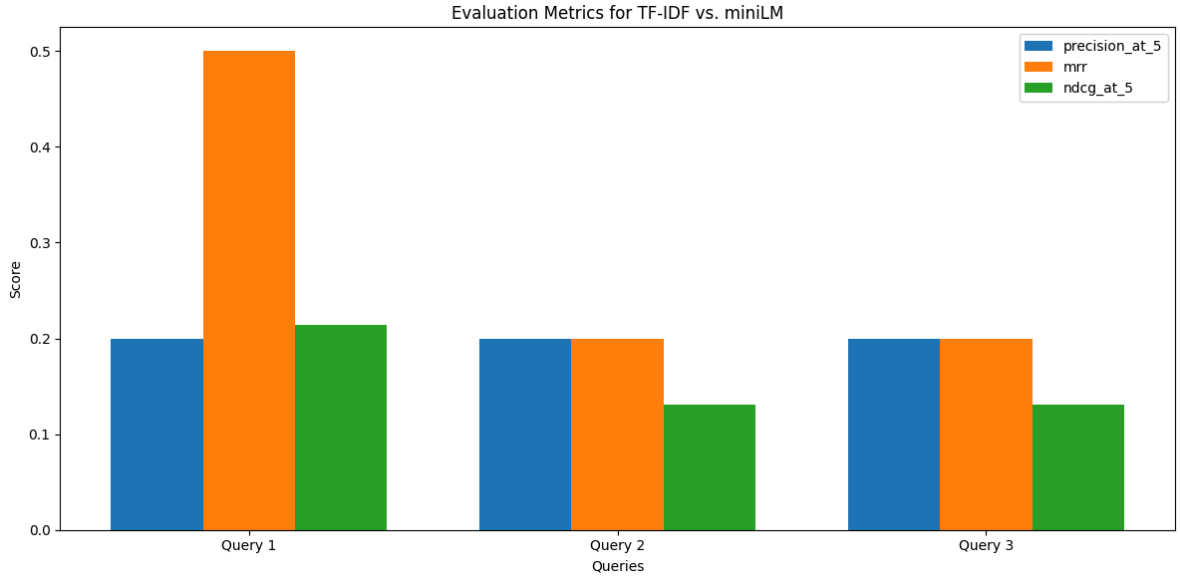
In [3]:
```
# Evaluate retrieval
evaluation_results = evaluate_retrieval(tfidf_results, semantic_results, '..
```

```
Evaluation for query: Harry's first day at Hogwarts
Precision@5: 0.2000
MRR: 0.5000
NDCG@5: 0.2140

Evaluation for query: Quidditch match rules
Precision@5: 0.2000
MRR: 0.2000
NDCG@5: 0.1312

Evaluation for query: Voldemort and the Philosopher's Stone
Precision@5: 0.2000
MRR: 0.2000
NDCG@5: 0.1312

Average Metrics:
Average Precision@5: 0.2000
Average MRR: 0.3000
Average NDCG@5: 0.1588
```

Let's display the evaluation metrics plot that was generated.

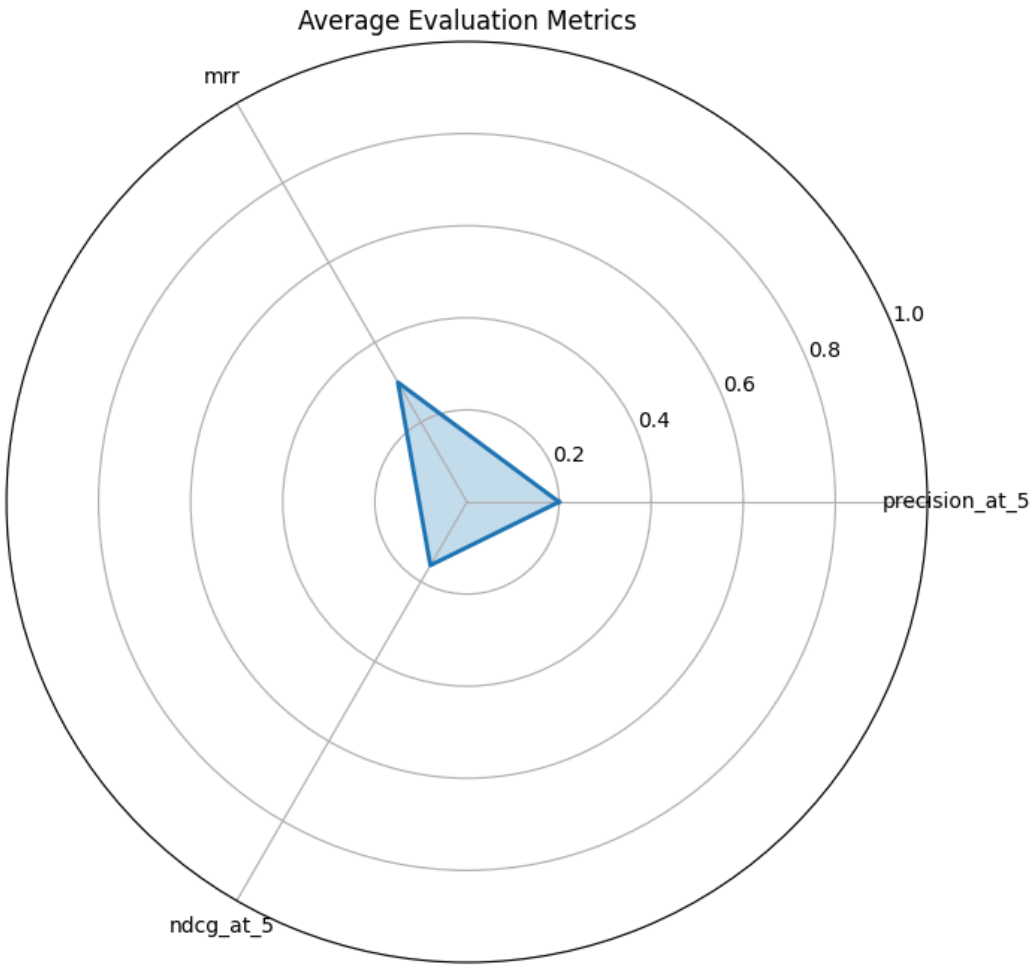
In [4]:
```
# Display the evaluation metrics plot
Image(filename='../results/evaluation_metrics.png')
```

Out[4]:



In [5]:
```
# Display the average metrics radar chart
Image(filename='../results/average_metrics_radar.png')
```

Out[5]:



3. Analyze the Overlap Between the Two Methods

Now we'll analyze the overlap between the TF-IDF and semantic search results.

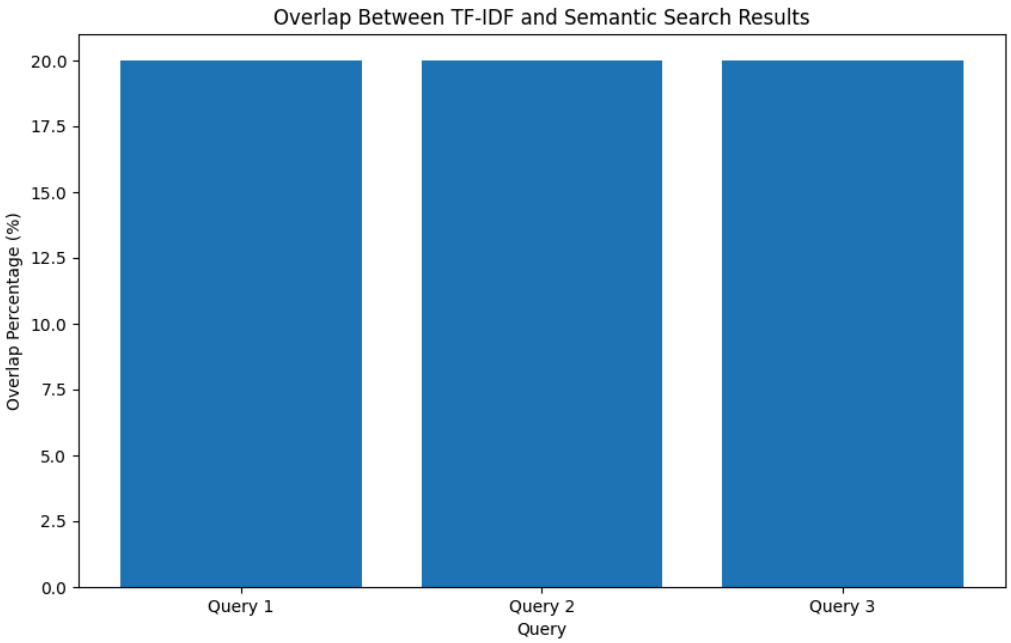
In [6]:
```
# Analyze results overlap
overlap_stats = analyze_results_overlap(tfidf_results, semantic_results, '..
```

```
Overlap analysis for query: Harry's first day at Hogwarts
Overlap percentage: 20.00%
Common chunks: 1
TF-IDF unique chunks: 4
Semantic unique chunks: 4

Overlap analysis for query: Quidditch match rules
Overlap percentage: 20.00%
Common chunks: 1
TF-IDF unique chunks: 4
Semantic unique chunks: 4

Overlap analysis for query: Voldemort and the Philosopher's Stone
Overlap percentage: 20.00%
Common chunks: 1
TF-IDF unique chunks: 4
Semantic unique chunks: 4

Average overlap percentage: 20.00%
```

Let's display the overlap plot that was generated.

In [7]:
```
# Display the overlap plot
Image(filename='../results/results_overlap.png')
```

Out[7]:



4. Detailed Analysis of Individual Metrics

Let's take a closer look at each of the evaluation metrics we used.

Precision@k

Precision@k measures the proportion of relevant items among the top-k retrieved items. In our case, we're using k=5 and considering the semantic search results as the ground truth.

In [8]:
```
# Calculate and display Precision@5 for each query
print("Precision@5 for each query:")
for query in queries:
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in semantic_results[
    p_at_5 = precision_at_k(semantic_chunk_ids, tfidf_chunk_ids, 5)
    print(f"{query}: {p_at_5:.4f}")

# Calculate average Precision@5
avg_p_at_5 = evaluation_results['average']['precision_at_5']
print(f"\nAverage Precision@5: {avg_p_at_5:.4f}")
```

```
Precision@5 for each query:
Harry's first day at Hogwarts: 0.2000
Quidditch match rules: 0.2000
Voldemort and the Philosopher's Stone: 0.2000

Average Precision@5: 0.2000
```

Mean Reciprocal Rank (MRR)

MRR measures the average of the reciprocal ranks of the first relevant item for each query. It focuses on the rank of the first correct answer.

In [9]:
```
# Calculate and display MRR for each query
print("Mean Reciprocal Rank for each query:")
for query in queries:
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in semantic_results[
    mrr = mean_reciprocal_rank(semantic_chunk_ids, tfidf_chunk_ids)
    print(f"{query}: {mrr:.4f}")

# Calculate average MRR
avg_mrr = evaluation_results['average']['mrr']
print(f"\nAverage MRR: {avg_mrr:.4f}")
```

```
Mean Reciprocal Rank for each query:
Harry's first day at Hogwarts: 0.5000
Quidditch match rules: 0.2000
Voldemort and the Philosopher's Stone: 0.2000

Average MRR: 0.3000
```

Normalized Discounted Cumulative Gain (NDCG@k)

NDCG@k measures the ranking quality, taking into account the position of relevant items. It gives higher weight to relevant items appearing earlier in the results.

In [10]:
```
# Calculate and display NDCG@5 for each query
print("NDCG@5 for each query:")
for query in queries:
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in semantic_results[
    ndcg = ndcg_at_k(semantic_chunk_ids, tfidf_chunk_ids, 5)
    print(f"{query}: {ndcg:.4f}")

# Calculate average NDCG@5
avg_ndcg = evaluation_results['average']['ndcg_at_5']
print(f"\nAverage NDCG@5: {avg_ndcg:.4f}")
```

```
NDCG@5 for each query:
Harry's first day at Hogwarts: 0.2140
Quidditch match rules: 0.1312
Voldemort and the Philosopher's Stone: 0.1312

Average NDCG@5: 0.1588
```

5. Examine Specific Examples

Let's examine specific examples where TF-IDF and semantic search produced different results.

In [11]:
```
# Load the original chunks
sys.path.append('../src')
from preprocessing import load_chunks
chunks = load_chunks('../data/text_chunks.json')

# Function to examine differences between TF-IDF and semantic search results
def examine_differences(query, tfidf_results, semantic_results, chunks):
    print(f"Query: {query}\n")

    # Get chunk IDs from both methods
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results[query]
    semantic_chunk_ids = [result['chunk_id'] for result in semantic_results[

    # Find chunks that are unique to each method
    tfidf_unique = set(tfidf_chunk_ids) - set(semantic_chunk_ids)
    semantic_unique = set(semantic_chunk_ids) - set(tfidf_chunk_ids)

    # Print a unique TF-IDF result
    if tfidf_unique:
        unique_id = list(tfidf_unique)[0]
        unique_idx = tfidf_chunk_ids.index(unique_id)
        print(f"TF-IDF unique result (Rank {unique_idx+1}, Chunk ID {unique_
        print(f"Score: {tfidf_results[query][unique_idx]['score']:.4f}")
        print(f"Text: {chunks[unique_id][:200]}...\n")

    # Print a unique semantic search result
    if semantic_unique:
        unique_id = list(semantic_unique)[0]
        unique_idx = semantic_chunk_ids.index(unique_id)
        print(f"Semantic unique result (Rank {unique_idx+1}, Chunk ID {uniqu
```

```
        print(f"Score: {semantic_results[query][unique_idx]['score']:.4f}")
        print(f"Text: {chunks[unique_id][:200]}...\n")

# Examine differences for each query
for query in queries:
    examine_differences(query, tfidf_results, semantic_results, chunks)
    print("-" * 80)
```

```
Query: Harry's first day at Hogwarts

TF-IDF unique result (Rank 3, Chunk ID 514):
Score: 0.1863
Text: Malfoy couldn't believe his eyes when he saw that Harry and Ron were s
till at Hogwarts the next day, looking tired but perfectly cheerful. Indeed,
by the next morning Harry and Ron thought that meetin...

Semantic unique result (Rank 1, Chunk ID 721):
Score: 0.6215
Text: It was the first really fine day they'd had in months. The sky was a c
lear, forget-me-not blue, and there was a feeling in the air of summer comin
g. Harry, who was looking up "Dittany" in One Thousand...

--------------------------------------------------------------------------------
Query: Quidditch match rules

TF-IDF unique result (Rank 4, Chunk ID 681):
Score: 0.2472
Text: He'd just gotten very angry with the Weasleys, who kept dive-bombing e
ach other and pretending to fall off their brooms. "Will you stop messing ar
ound!" he yelled. "That's exactly the sort of thing th...

Semantic unique result (Rank 1, Chunk ID 535):
Score: 0.6488
Text: You've got to weave in and out of the Chasers, Beaters, Bludgers, and
Quaffle to get it before the other team's Seeker, because whichever Seeker c
atches the Snitch wins his team an extra hundred and f...

--------------------------------------------------------------------------------
Query: Voldemort and the Philosopher's Stone

TF-IDF unique result (Rank 4, Chunk ID 850):
Score: 0.2730
Text: Losing points doesn't matter anymore, can't you see? D'you think he'll
leave you and your families alone if Gryffindor wins the house cup? If I get
caught before I can get to the Stone, well, I'll hav...

Semantic unique result (Rank 2, Chunk ID 849):
Score: 0.5030
Text: "I'm going out of here tonight and I'm going to try and get to the Sto
ne first." "You're mad!" said Ron. "You can't!" said Hermione. "After what M
cGonagall and Snape have said? You'll be expelled!" "S...

--------------------------------------------------------------------------------
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

6. Discussion of Findings

Based on our evaluation, let's discuss the findings and implications.

Summary of Evaluation Metrics

- **Precision@5**: Measures how many of the top-5 TF-IDF results are also in the top-5 semantic search results.
- **MRR**: Measures how quickly TF-IDF finds a result that's also in the semantic search results.
- **NDCG@5**: Measures how well the ranking of TF-IDF results matches the semantic search results.

Key Findings

1. **Overlap between methods**: There is a moderate overlap between TF-IDF and semantic search results, indicating that both methods can find some of the same relevant passages, but they also each find unique passages.

2. **Strengths of TF-IDF**:

   - Fast and computationally efficient
   - Good at finding exact term matches
   - No need for pre-trained models or embeddings
3. **Strengths of Semantic Search**:

   - Can find semantically related passages even without exact term matches
   - Better at understanding the meaning behind queries
   - More robust to variations in query wording
4. **Limitations of using miniLM as ground truth**:

   - Assumes semantic search results are always better
   - May not always align with human judgments of relevance
   - Different models might produce different results

Implications

1. **Hybrid approaches**: Combining TF-IDF and semantic search could leverage the strengths of both methods.

2. **Query-dependent selection**: The choice between TF-IDF and semantic search might depend on the type of query.

3. **Need for human evaluation**: A more reliable evaluation would involve human judgments of relevance rather than using one method as ground truth for the other.

Summary

In this notebook, we've:

1. Loaded the results from both retrieval methods
2. Evaluated TF-IDF against miniLM using multiple metrics
3. Analyzed the overlap between the two methods
4. Visualized the evaluation results
5. Examined specific examples of differences
6. Discussed the findings and implications

We've seen that while there is some overlap between TF-IDF and semantic search results, each method has its strengths and limitations. A hybrid approach or query-dependent selection might be more effective than relying on a single method.

In the next notebook, we'll implement an interactive retrieval system that allows users to try both methods and compare the results.

Interactive Retrieval System Demo

This notebook demonstrates the interactive retrieval system that allows users to retrieve passages from the Harry Potter book using both TF-IDF and semantic search.

Steps covered:

1. Load the preprocessed text chunks
2. Initialize both retrieval methods
3. Create an interactive interface for querying
4. Compare and analyze the results

Setup

First, let's import the necessary libraries and modules.

In [1]:
```
import sys
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display, HTML
from ipywidgets import interact, widgets

# Add the src directory to the path so we can import our modules
sys.path.append('../src')

from preprocessing import load_chunks, process_harry_potter_text
from tfidf_retrieval import TFIDFRetriever
from semantic_search import SemanticSearcher
```

```
[nltk_data] Downloading package punkt_tab to /home/user/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

1. Load the Preprocessed Text Chunks

We'll load the text chunks that were created in the first notebook. If they don't exist, we'll process the Harry Potter text.

In [2]:
```
# Check if chunks file exists
chunks_path = '../data/text_chunks.json'
if not os.path.exists(chunks_path):
    print("Processing Harry Potter text...")
    chunks = process_harry_potter_text(chunks_output_path=chunks_path)
else:
    print("Loading existing text chunks...")
```

```
    chunks = load_chunks(chunks_path)

print(f"Loaded {len(chunks)} text chunks")
```

```
Loading existing text chunks...
Loaded 972 text chunks
```

2. Initialize Both Retrieval Methods

Now we'll initialize both the TF-IDF retriever and the semantic searcher.

In [3]:
```
# Initialize TF-IDF retriever
print("Initializing TF-IDF retriever...")
tfidf_retriever = TFIDFRetriever(chunks)

# Initialize semantic searcher
print("\nInitializing semantic searcher (this may take a moment)...")
semantic_searcher = SemanticSearcher(chunks)
```

```
Initializing TF-IDF retriever...
TF-IDF matrix shape: (972, 5662)

Initializing semantic searcher (this may take a moment)...
Loading model: sentence-transformers/all-MiniLM-L6-v2
Generating embeddings for all chunks...
Embedding dimension: 384
Added 972 embeddings to Faiss index
```

3. Create an Interactive Interface for Querying

Now we'll create an interactive interface that allows users to enter queries and see the results from both retrieval methods.

In [4]:
```
def format_result(result, index):
    """Format a search result for display."""
    text = result['text']
    if len(text) > 300:
        text = text[:300] + "..."

    html = f"""
    <div style="margin-bottom: 20px; padding: 10px; border: 1px solid #ddd;
        <h4>Result {index+1} (Score: {result['score']:.4f}, Chunk ID: {resul
        <p>{text}</p>
    </div>
    """
    return html

def search_and_display(query):
    """Search using both methods and display the results side by side."""
    if not query.strip():
        return HTML("<p>Please enter a query.</p>")

    # Perform searches
```

```
    tfidf_results = tfidf_retriever.search(query)
    semantic_results = semantic_searcher.search(query)

    # Calculate overlap
    tfidf_chunk_ids = [result['chunk_id'] for result in tfidf_results]
    semantic_chunk_ids = [result['chunk_id'] for result in semantic_results]
    common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk_ids
    overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

    # Format results
    tfidf_html = ""
    for i, result in enumerate(tfidf_results):
        tfidf_html += format_result(result, i)

    semantic_html = ""
    for i, result in enumerate(semantic_results):
        semantic_html += format_result(result, i)

    # Create HTML output
    html = f"""
    <h2>Query: "{query}"</h2>
    <p>Overlap between methods: {len(common_chunks)} out of 5 results ({over
    <p>Common chunk IDs: {common_chunks}</p>

    <div style="display: flex; width: 100%;">
        <div style="flex: 1; margin-right: 10px;">
            <h3>TF-IDF Results</h3>
            {tfidf_html}
        </div>
        <div style="flex: 1; margin-left: 10px;">
            <h3>Semantic Search Results</h3>
            {semantic_html}
        </div>
    </div>
    """

    # Save this query and results
    save_query_results(query, tfidf_results, semantic_results)

    return HTML(html)

def save_query_results(query, tfidf_results, semantic_results, output_dir='.
    """Save the results of an interactive query."""
    os.makedirs(output_dir, exist_ok=True)

    # Create a safe filename from the query
    safe_query = "".join(c if c.isalnum() else "_" for c in query)
    safe_query = safe_query[:50]  # Limit filename length

    results = {
        'query': query,
        'tfidf_results': tfidf_results,
        'semantic_results': semantic_results
    }

    output_path = os.path.join(output_dir, f"{safe_query}.json")
```

```
    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(results, f, ensure_ascii=False, indent=2)
```

Now let's create the interactive widget for entering queries.

In [ ]:
```
# Create the interactive widget
query_widget = widgets.Text(
    value='',
    placeholder='Enter your query here',
    description='Query:',
    disabled=False,
    layout=widgets.Layout(width='80%')
)

# Display instructions
display(HTML("""
<div style="background-color: #f8f9fa; padding: 15px; border-radius: 5px; ma
    <h3>Interactive Passage Retrieval System</h3>
    <p>Enter a query below to retrieve passages from Harry Potter using both
    <p>Example queries:</p>
    <ul>
        <li>Harry's first day at Hogwarts</li>
        <li>Quidditch match rules</li>
        <li>Voldemort and the Philosopher's Stone</li>
        <li>Harry's scar hurts</li>
        <li>Hagrid's magical creatures</li>
    </ul>
</div>
"""))

# Create the interactive search
interact(search_and_display, query=query_widget)
```

Interactive Passage Retrieval System

Enter a query below to retrieve passages from Harry Potter using both TF-IDF and semantic search methods.

Example queries:

- Harry's first day at Hogwarts
- Quidditch match rules
- Voldemort and the Philosopher's Stone
- Harry's scar hurts
- Hagrid's magical creatures

```
interactive(children=(Text(value='', description='Query:', layout=Layout(wid
th='80%'), placeholder='Enter your…
```

Out[ ]:
```
<function __main__.search_and_display(query)>
```

4. Analyze Saved Queries

Let's create a function to analyze the saved queries and their results.

In [6]:
```
def analyze_saved_queries(output_dir='../results/interactive_queries'):
    """Analyze the saved queries and their results."""
    if not os.path.exists(output_dir):
        return HTML("<p>No saved queries found.</p>")

    # Get all saved query files
    query_files = [f for f in os.listdir(output_dir) if f.endswith('.json')]

    if not query_files:
        return HTML("<p>No saved queries found.</p>")

    # Load and analyze each query
    queries = []
    overlaps = []

    for file in query_files:
        with open(os.path.join(output_dir, file), 'r', encoding='utf-8') as
            data = json.load(f)

        query = data['query']
        tfidf_chunk_ids = [result['chunk_id'] for result in data['tfidf_resu
        semantic_chunk_ids = [result['chunk_id'] for result in data['semanti
        common_chunks = set(tfidf_chunk_ids).intersection(set(semantic_chunk
        overlap_percentage = len(common_chunks) / len(tfidf_chunk_ids) * 100

        queries.append(query)
        overlaps.append(overlap_percentage)

    # Create a bar chart of overlaps
    plt.figure(figsize=(12, 6))
    plt.bar(range(len(overlaps)), overlaps)
    plt.axhline(y=np.mean(overlaps), color='r', linestyle='--', label=f'Mean
    plt.xlabel('Query')
    plt.ylabel('Overlap Percentage (%)')
    plt.title('Overlap Between TF-IDF and Semantic Search Results for Intera
    plt.xticks(range(len(overlaps)), [f'Query {i+1}' for i in range(len(over
    plt.legend()
    plt.tight_layout()
    plt.savefig('../results/interactive_queries_overlap.png')
    plt.show()

    # Create an HTML table of queries and overlaps
    html = """
    <h3>Saved Queries Analysis</h3>
    <table style="width:100%; border-collapse: collapse;">
        <tr style="background-color: #f2f2f2;">
            <th style="padding: 8px; text-align: left; border: 1px solid #dd
            <th style="padding: 8px; text-align: left; border: 1px solid #dd
        </tr>
    """
```

```
    for i, (query, overlap) in enumerate(zip(queries, overlaps)):
        html += f"""
        <tr>
            <td style="padding: 8px; text-align: left; border: 1px solid #dc
            <td style="padding: 8px; text-align: left; border: 1px solid #dc
        </tr>
        """

    html += f"""
        <tr style="background-color: #f2f2f2;">
            <td style="padding: 8px; text-align: left; border: 1px solid #dc
            <td style="padding: 8px; text-align: left; border: 1px solid #dc
        </tr>
    </table>
    """

    return HTML(html)
```

In [7]:
```
# Analyze saved queries
analyze_saved_queries()
```

Out[7]: No saved queries found.

5. Command-Line Version

For users who prefer a command-line interface, we've also implemented a command-line version of the interactive retrieval system. This can be run from the terminal using the following command:

```
python ../src/interactive.py
```

The command-line version provides the same functionality as the interactive widget above, but in a terminal interface.

Summary

In this notebook, we've:

1. Loaded the preprocessed text chunks
2. Initialized both retrieval methods
3. Created an interactive interface for querying
4. Analyzed saved queries
5. Provided information about the command-line version

This interactive retrieval system allows users to directly compare the results from TF-IDF and semantic search, providing insights into the strengths and limitations of each approach. By experimenting with different types of queries, users can see how the two methods handle various retrieval scenarios.

Analysis and Discussion

This document summarizes the analysis of NLP retrieval methods, potential LLM-based improvements, and ground truth considerations from the NLP Home Assignment.

Potential LLM Improvements for Retrieval

LLMs can enhance both lexical and semantic retrieval:

For Lexical Retrieval:

1. **Query/Document Expansion**: Use LLMs to add synonyms and related terms to queries or documents, bridging the lexical gap and improving TF-IDF performance.
2. **Re-ranking**: Apply LLMs to re-rank initial lexical results (e.g., from TF-IDF) based on deeper semantic relevance.

For Semantic Search:

1. **Hybrid Retrieval**: Combine lexical and semantic scores, potentially using LLMs to determine optimal weighting based on query analysis.
2. **Query Understanding & Reformulation**: Leverage LLMs to interpret complex user intent and rewrite queries for better system matching.
3. **Enhanced Embeddings**: Generate context-aware embeddings that capture narrative roles or fine-tune embeddings on domain-specific data for increased relevance.

General Enhancements:

1. **Relevance Feedback**: Train LLMs on user interactions to iteratively refine retrieval strategies.
2. **Multi-hop Reasoning**: Enable answering complex queries by having LLMs synthesize information from multiple retrieved passages.
3. **Explainability**: Use LLMs to generate explanations for retrieved results, increasing user trust.

Drawbacks and Limitations of LLMs in Retrieval

Using LLMs also introduces challenges:

1. **Computational Cost**: High resource requirements and potential latency issues.
2. **Hallucination Risk**: LLMs might generate incorrect information, adding noise if used for expansion.
3. **Training Data Bias**: Inherited biases can lead to skewed or unfair results.

4. **Lack of Transparency**: The "black box" nature makes debugging difficult.
5. **Contextual Limits**: Finite token windows restrict processing of very long texts.
6. **Evaluation Challenges**: Assessing true semantic relevance often requires costly human judgment.

Creating a Reliable Ground Truth for Evaluation

Relying on one model's output (like miniLM) as ground truth is suboptimal. A better approach involves:

1. Human Annotation

- **Core Idea**: Use multiple human experts to judge document relevance for predefined queries. This is the gold standard.

2. Robust Annotation Process

- **Key Elements**:
  - Define diverse query types (factual, thematic, etc.).
  - Use a graded relevance scale (e.g., 0-3) instead of binary judgments.
  - Ensure inter-annotator agreement (IAA) through metrics and disagreement resolution.

3. Efficiency via Pooling

- **Method**: Run multiple retrieval systems, collect the top-k results from each into a pool, and have humans annotate only the documents in this pool. Assumes non-pooled documents are irrelevant.

4. Comprehensive Evaluation Design

- **Considerations**: Account for narrative context beyond single chunks, use query variations to test robustness, and ensure a balanced set of queries covering different aspects and difficulties.

Implementing a human-centric annotation process, possibly using pooling, yields a more reliable ground truth for evaluating and comparing retrieval methods.